# SC1001: Introduction to Computational Thinking

## Week 12: Algorithm Design

Jordan Boyd-Graber

Nanyang Technological University

# Slides

Slides and updates will be posted at

`boydgraber.org/teaching/SC_1001/`

# EMNLP Best Paper

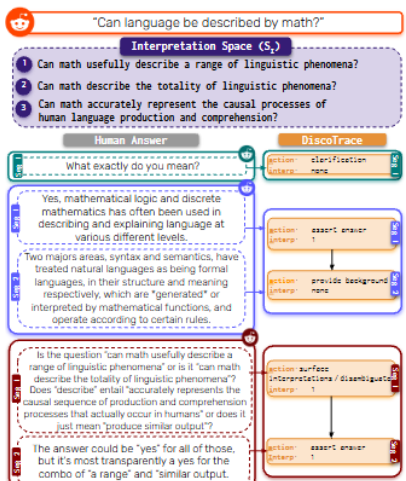
I missed the start of class because our paper won a Best Paper Award at EMNLP.




- The paper studies how language can be described by math.
- The method compares human answers with model reasoning traces.
- This is also algorithm design: define the representation, then reason about the steps.

# EMNLP Best Paper

I missed the start of class because our paper won a Best Paper Award at EMNLP.




- The paper studies how language can be described by math.
- The method compares human answers with model reasoning traces.
- This is also algorithm design: define the representation, then reason about the steps.

# Overview

1. Compare bubble sort and cocktail shaker sort
2. Use loop invariants to improve bounds
3. Separate useful examples from worst-case behavior
4. Add patience sorting as another quadratic worst case

# Cocktail Shaker Sort

Cocktail shaker sort alternates directions.

- A left-to-right pass moves large items toward the end.
- A right-to-left pass moves small items toward the beginning.
- The algorithm stops when a full pass makes no swaps.

| Step | Pass | List |
|---|---|---|
| Start | input | `[5, 9, 4, 6, 2, 7]` |

Start with the unsorted list.

# Cocktail Shaker Sort

Cocktail shaker sort alternates directions.

- A left-to-right pass moves large items toward the end.
- A right-to-left pass moves small items toward the beginning.
- The algorithm stops when a full pass makes no swaps.

| Step | Pass | List |
|---|---|---|
| Start | input | `[5, 9, 4, 6, 2, 7]` |
| 1 | left to right | `[5, 4, 6, 2, 7, 9]` |

The first left-to-right pass moves 9 to its final position; 2 moves left but still needs the backward pass.

# Cocktail Shaker Sort

Cocktail shaker sort alternates directions.

- A left-to-right pass moves large items toward the end.
- A right-to-left pass moves small items toward the beginning.
- The algorithm stops when a full pass makes no swaps.

| Step | Pass | List |
|---|---|---|
| Start | input | `[5, 9, 4, 6, 2, 7]` |
| 1 | left to right | `[5, 4, 6, 2, 7, 9]` |
| 2 | right to left | `[2, 5, 4, 6, 7, 9]` |

The first right-to-left pass moves 2 to its final position.

# Cocktail Shaker Sort

Cocktail shaker sort alternates directions.

- A left-to-right pass moves large items toward the end.
- A right-to-left pass moves small items toward the beginning.
- The algorithm stops when a full pass makes no swaps.

| Step | Pass | List |
|---|---|---|
| Start | input | `[5, 9, 4, 6, 2, 7]` |
| 1 | left to right | `[5, 4, 6, 2, 7, 9]` |
| 2 | right to left | `[2, 5, 4, 6, 7, 9]` |
| 3 | left to right | `[2, 4, 5, 6, 7, 9]` |

The next left-to-right pass moves 4 into place.

# Cocktail Shaker Sort

Cocktail shaker sort alternates directions.

- A left-to-right pass moves large items toward the end.
- A right-to-left pass moves small items toward the beginning.
- The algorithm stops when a full pass makes no swaps.

| Step | Pass | List |
|---|---|---|
| Start | input | `[5, 9, 4, 6, 2, 7]` |
| 1 | left to right | `[5, 4, 6, 2, 7, 9]` |
| 2 | right to left | `[2, 5, 4, 6, 7, 9]` |
| 3 | left to right | `[2, 4, 5, 6, 7, 9]` |
| 4 | no swaps | `[2, 4, 5, 6, 7, 9]` |

The next pass makes no swaps, so the algorithm stops.

# Problem 1: Python Code

Write Python code for cocktail shaker sort on an input list `values`.

Start from the bubble sort pattern, then add a right-to-left pass.

# Problem 3: Better Than Bubble Sort

Give an input list, or class of lists, where bubble sort is slow but cocktail shaker sort is asymptotically better.

# Problem 4: Worst Case

Argue that the worst-case time complexity of cocktail shaker sort is no better than $O(n^2)$.

Is this asymptotically better than bubble sort?

# Problem 5: Patience Sorting

Patience sorting uses piles, similar to a card game.

In the US, this game is usually called Solitaire.

For each item:

1. Put it on the leftmost pile whose top item is at least as large.
2. If no such pile exists, start a new pile.

After building the piles, repeatedly remove the smallest visible pile top to produce the sorted output.




Image: Maaaks, GPL, Wikimedia Commons, curid 6952483.

# Patience Sorting Demo

Use the same six-item list:

$$[5, 9, 4, 6, 2, 7].$$




Place 5 on the first pile.

# Patience Sorting Demo

Use the same six-item list:

$$[5, 9, 4, 6, 2, 7].$$

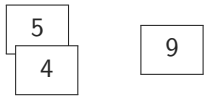



9 is larger than the first pile top, so it starts a new pile.

# Patience Sorting Demo

Use the same six-item list:

$$[5, 9, 4, 6, 2, 7].$$

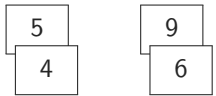



4 goes on the leftmost pile whose top is at least 4.

# Patience Sorting Demo

Use the same six-item list:

$$[5, 9, 4, 6, 2, 7].$$

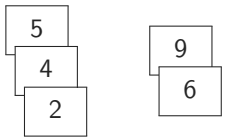



6 goes on 9, becoming the new visible top.

# Patience Sorting Demo

Use the same six-item list:

$$[5, 9, 4, 6, 2, 7].$$

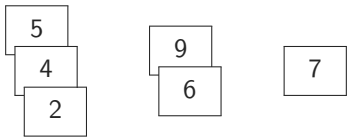



2 goes on the first pile, becoming the new visible top.

# Patience Sorting Demo

Use the same six-item list:

$$[5, 9, 4, 6, 2, 7].$$




7 starts a third pile.

# Problem 5: Discussion

Write Python code for the simple version of patience sorting.

- Scan pile tops from left to right to place each item.
- Scan pile tops to find the smallest visible item during output.
- Give a worst-case input class for this simple implementation.